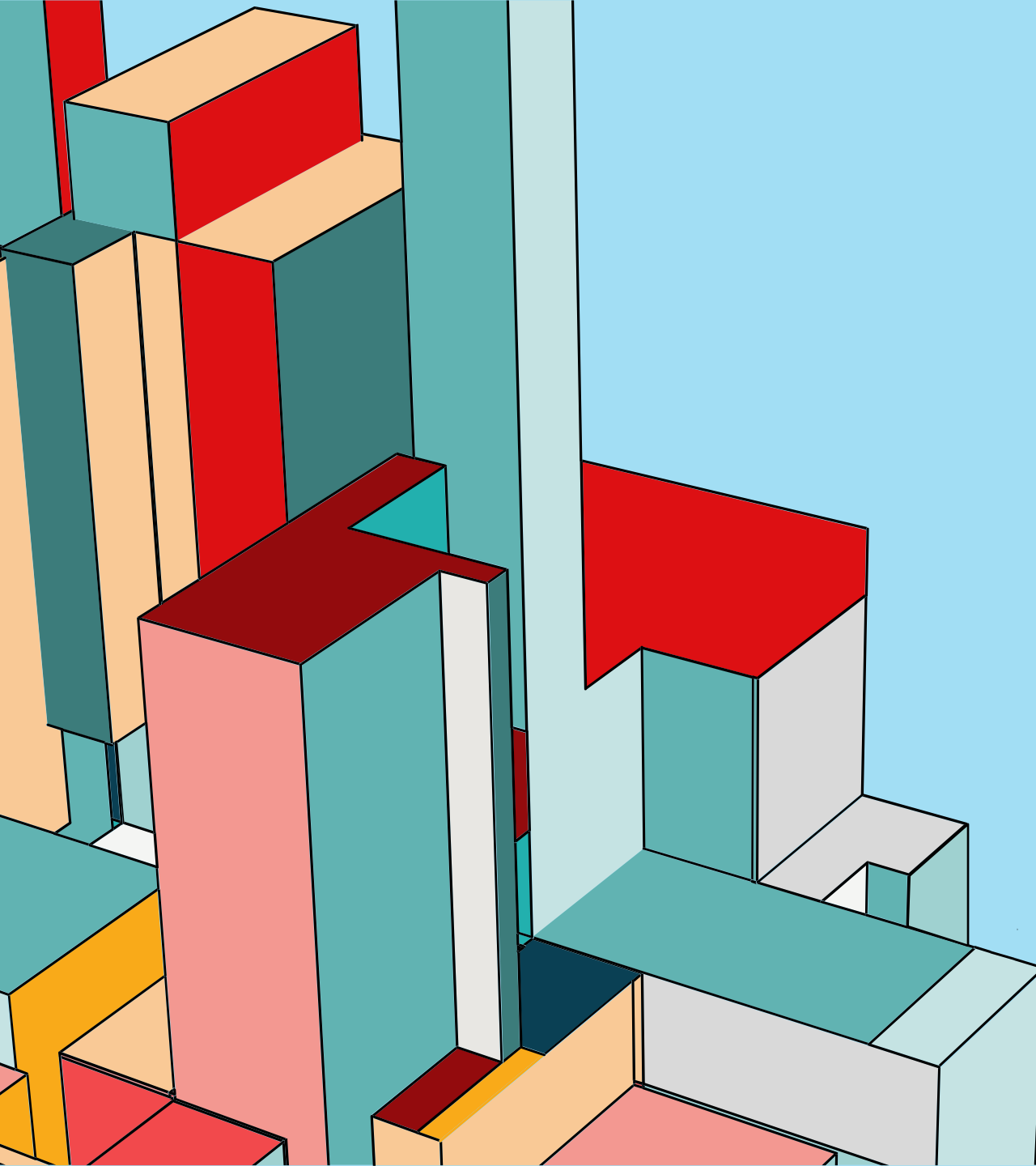


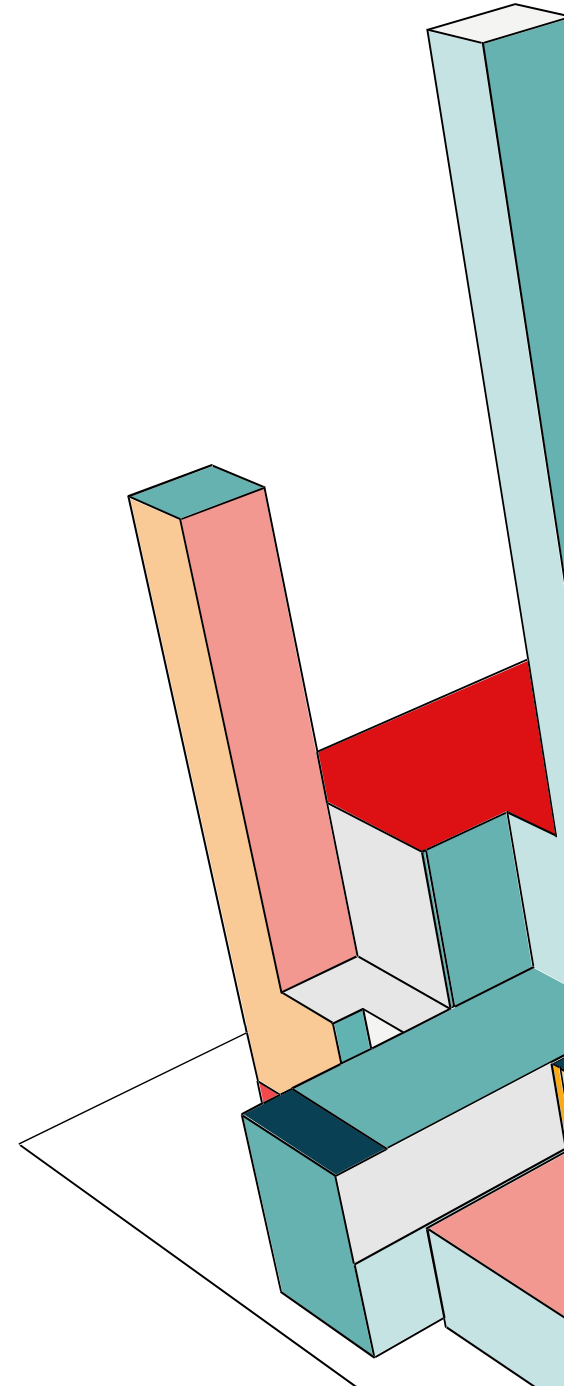


LECTURE 10 **MORE GREEDY ALGORITHMS**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

TODAY'S AGENDA

- Review of Exam 1
- Dijkstra's algorithm
- Huffman codes



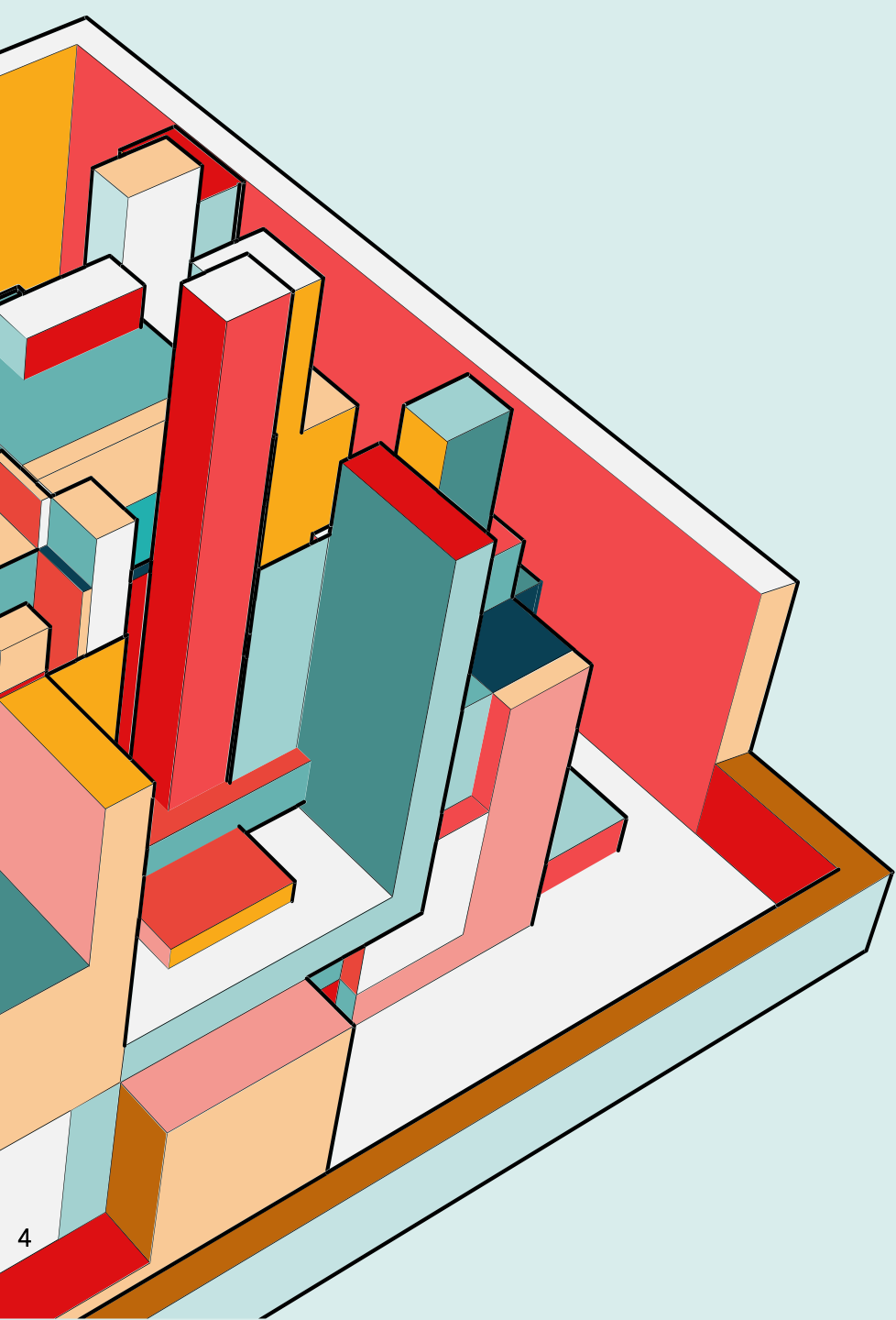
TAKEAWAYS FROM EXAM 1

Recursion. Recursion. Recursion.

Examples are not proof.

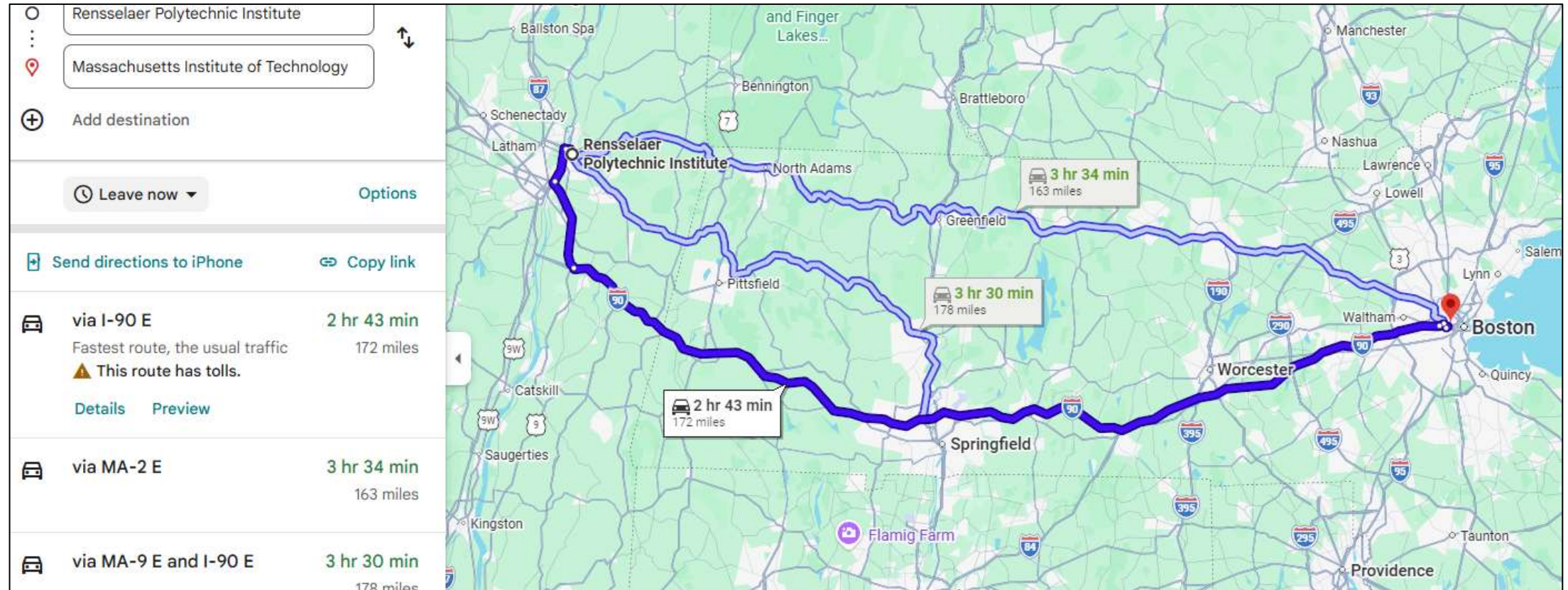
Master Theorem does not work on “decrease by k ” style recurrence relations.

Divide-**and-conquer** requires that you use the solutions to the subproblems.



DIJKSTRA'S ALGORITHM

NAVIGATION



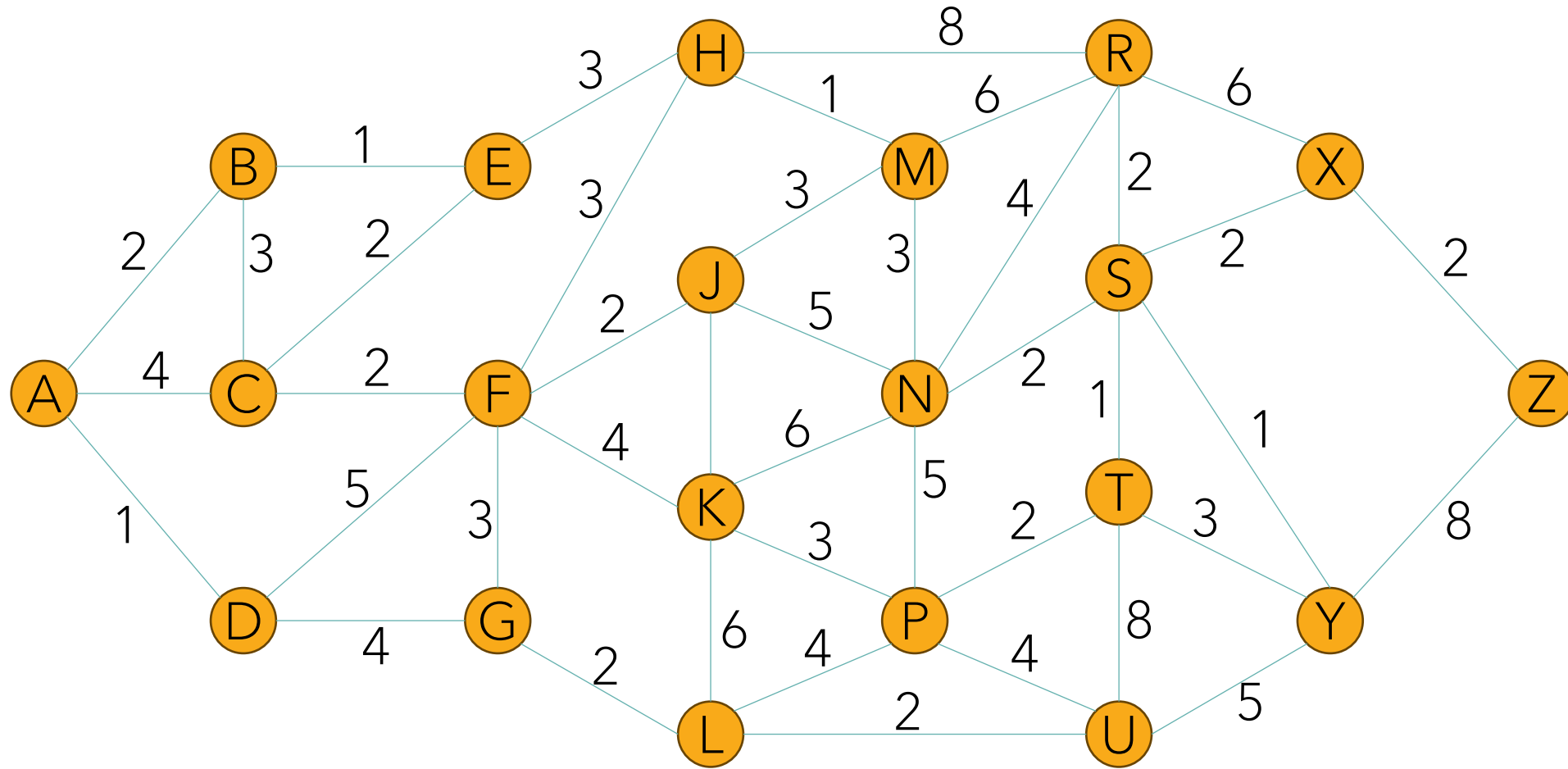
NAVIGATION IS FUNDAMENTALLY A GRAPH PROBLEM

Highway data is stored as a graph: Intersections are vertices and the roads between them are the edges.

For a simplified illustration of this, check out Jim Teresco's [METAL project](#).

Thus, the problem of finding the shortest route from one place to another reduces to the problem of finding the shortest path through a graph.

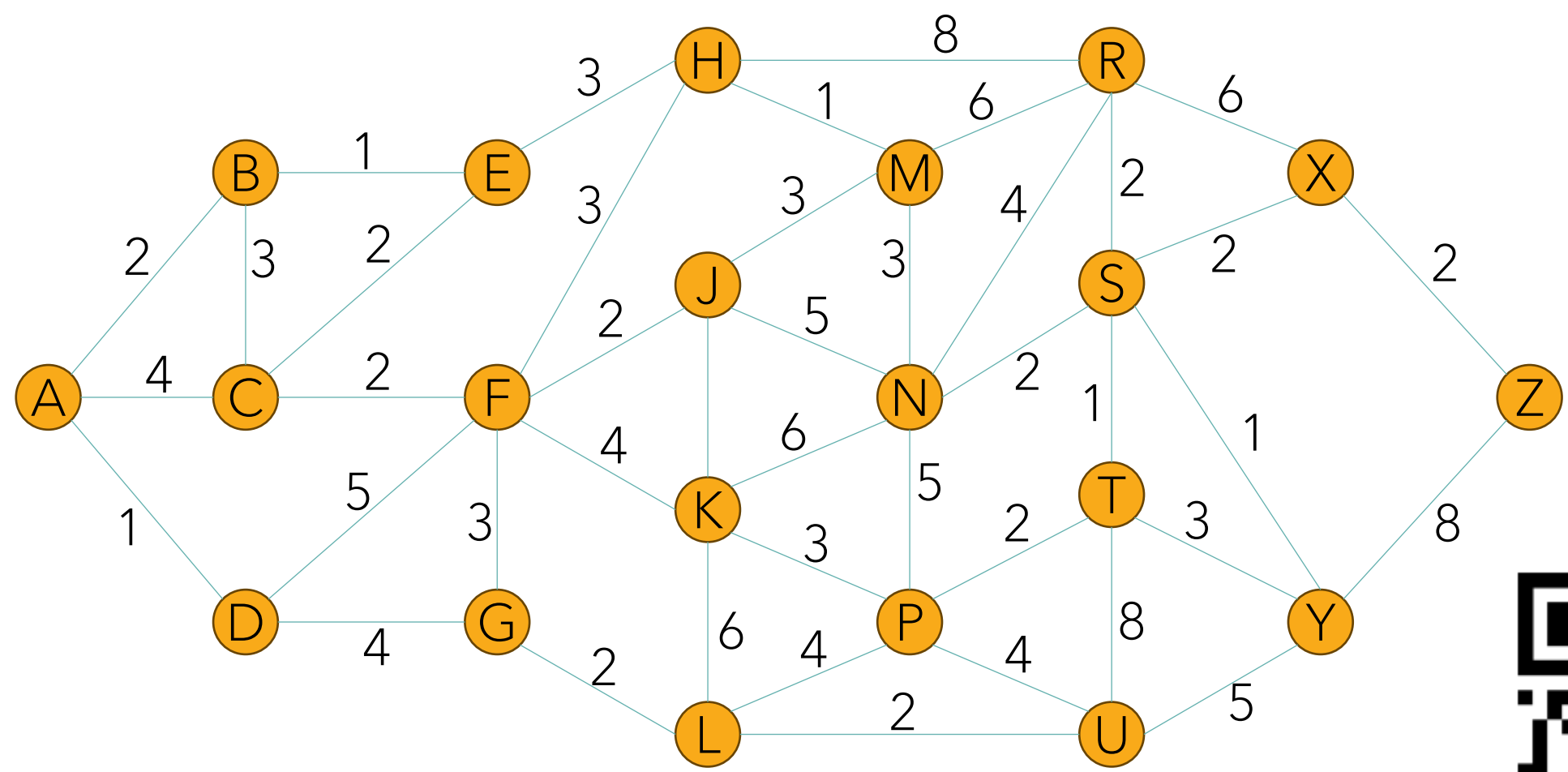
FROM A TO Z



What is the shortest path from A to Z?

More importantly, what is your algorithm for finding it?

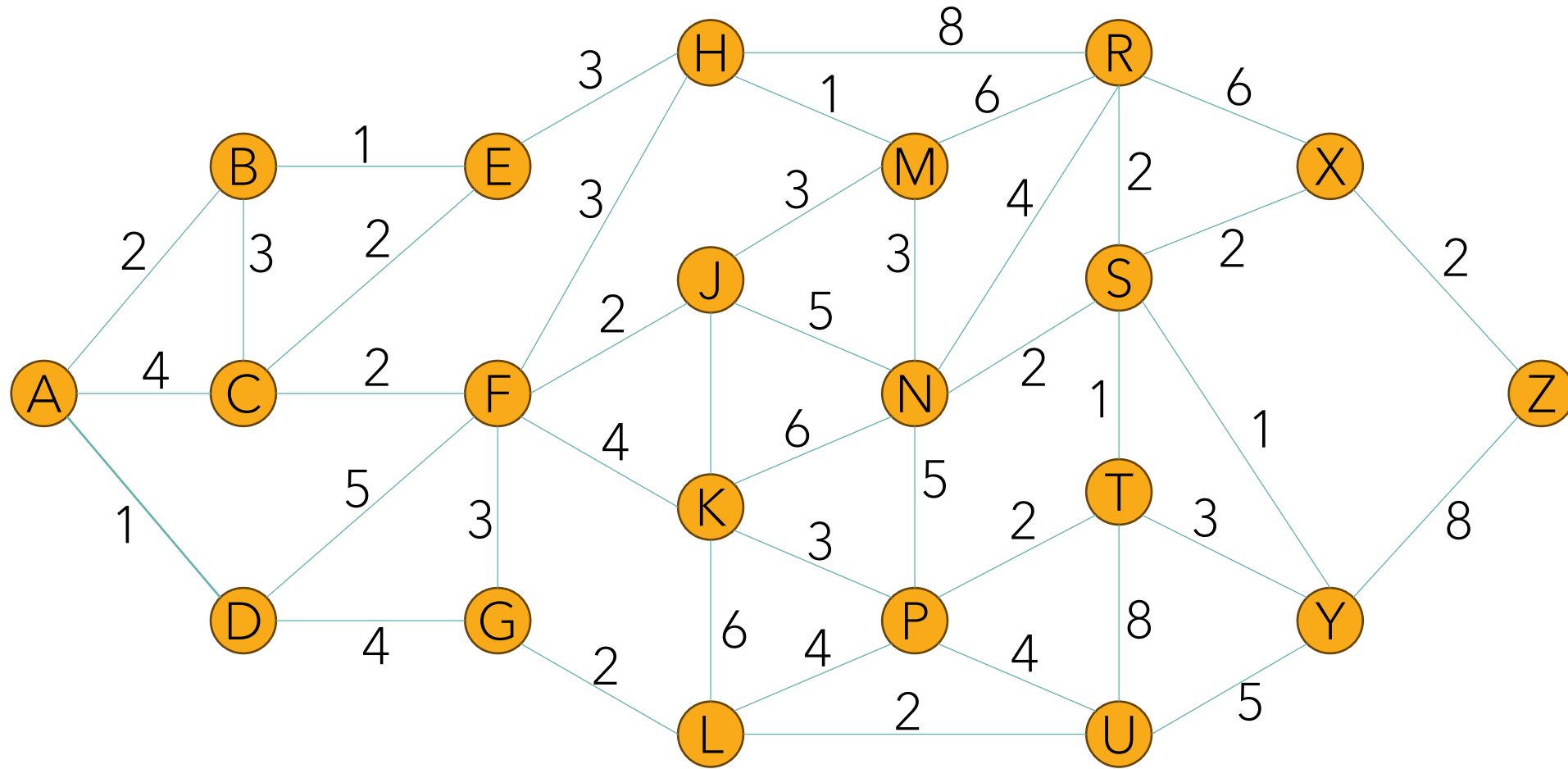
FROM A TO Z



Find the length of the shortest path from A to Z and enter it in the Submittity poll.

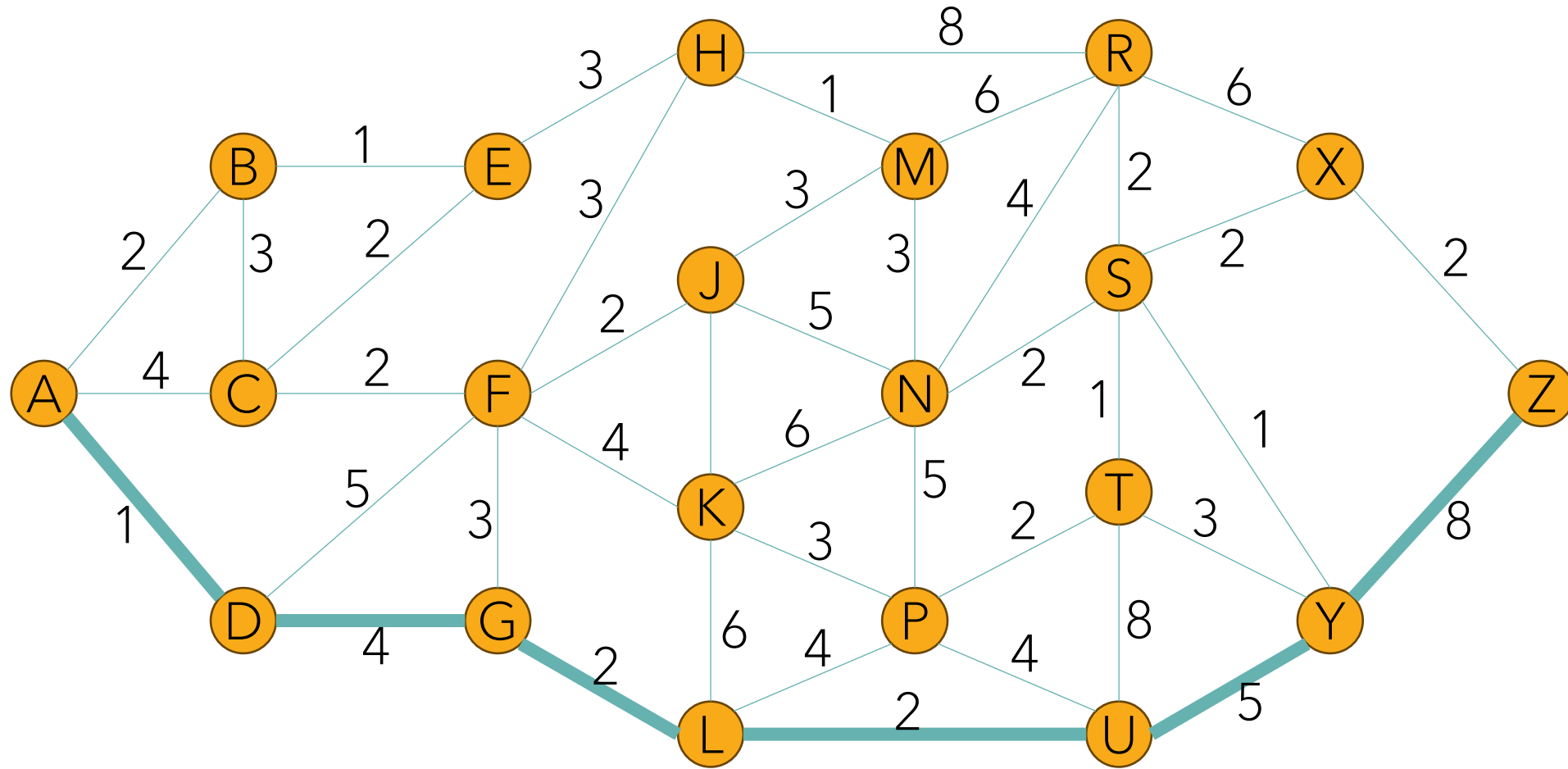


WHAT DOESN'T WORK



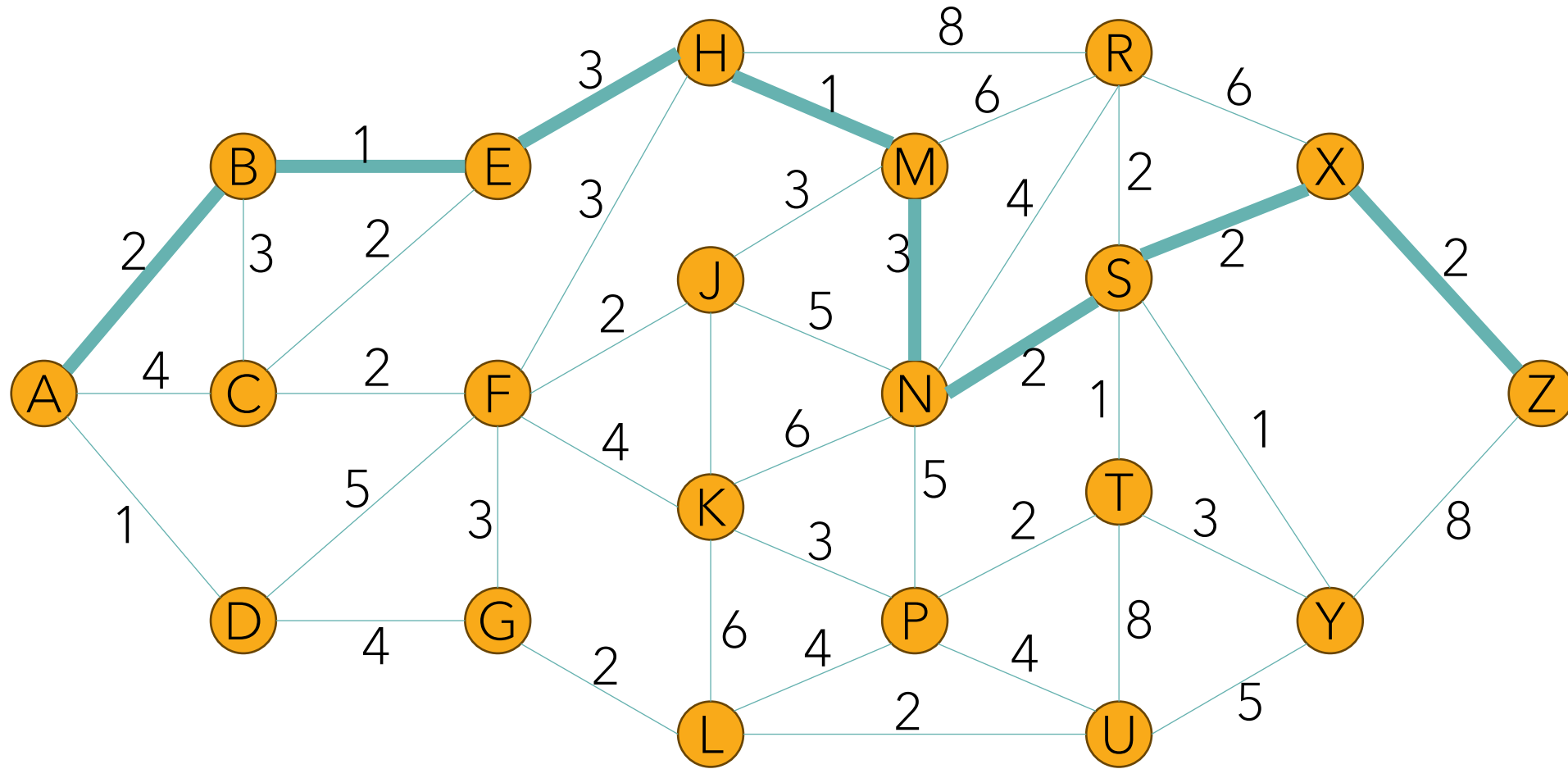
A naïve greedy approach would simply take the shortest link at every opportunity.

WHAT DOESN'T WORK



A naïve greedy approach would simply take the shortest link at every opportunity. Even if we can constrain this to “go in the correct direction”, this does not guarantee the shortest route.

FROM A TO Z



This is the shortest path – how can we find it?

Key insight: We don't actually know which nodes are closer to the target!

WEIGHTED GRAPH SEARCH

If we truly have no hints as to the “direction” of the target node, then we really don’t have much choice other than to just exhaustively search the graph until we find it.

We already have two algorithms for searching a graph, breadth-first search and depth-first search, but generally both of these are intended for unweighted graphs.

Can we modify one of these to work on weighted graphs? If so, which one should we choose?

AN ENHANCED BREADTH-FIRST SEARCH

The hint is in the lack of direction. Depth-first search would be ideal if we knew which way to look – it has the potential to reach the target node very quickly.

But without that guidance, DFS could spend ages exploring paths that are nowhere near the target.

Recall that breadth-first search explores nodes in order by distance from the source, where distance is measured in the number of links followed.

We can easily update this so that it uses the weights (e.g. distances) on the edges instead of just the count.

BREADTH-FIRST SEARCH REMINDER

```
def bfs( $G, v_1$ ):  
     $t = 1$ ; set  $\text{time}(v) = 0$  for all vertices  $v$  in  $G$   
     $\text{time}(v_1) = t$ ;  $\text{parent}(v_1) = \text{NULL}$ ; add  $v_1$  to the queue  
    while the queue is not empty:  
         $v =$  next vertex in queue (& remove from queue)  
        for each neighbor  $n$  of  $v$ :  
            if  $\text{time}(n) == 0$ :  
                 $t = t + 1$   
                 $\text{time}(n) = t$ ;  $\text{parent}(n) = v$   
                add  $n$  to the queue
```

We used a queue in the original BFS. This works because all distances are 1. Now we want to make sure we explore the next-closest node...

ONCE AGAIN, PRIORITY QUEUE TO THE RESCUE!

Just as in Prim's algorithm, we can use a priority queue to keep track of which vertex we should explore next.

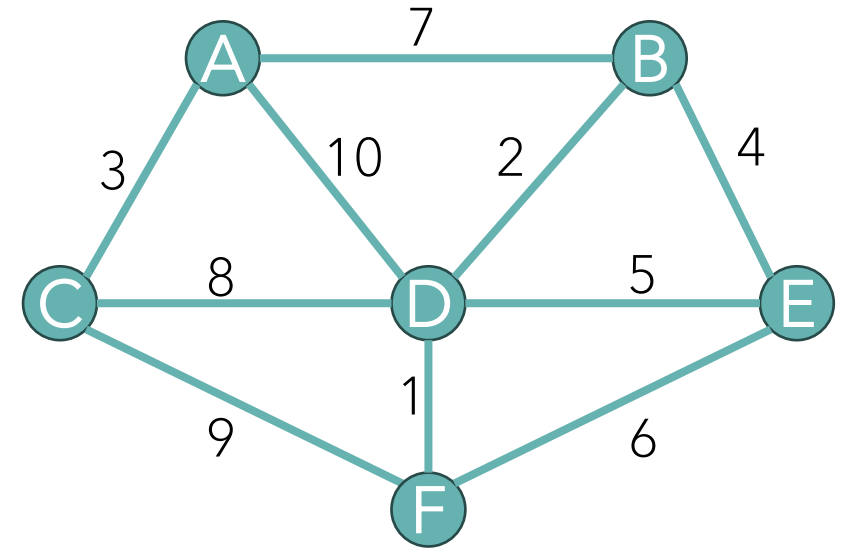
In fact, Dijkstra's algorithm (which is what we're describing here) is very similar to Prim's – the key difference is that the priorities in Prim's algorithm are just the next edge length, while the priorities in Dijkstra's algorithm are the distance from the source node.

A useful way to think of Dijkstra's algorithm is as a cross between breadth-first search and Prim's algorithm.

EXAMPLE RUN OF DIJKSTRA'S ALGORITHM

Here is the example graph from last class. We'll use it to demonstrate Dijkstra's algorithm, starting from A and targeting node F.

Initially, we put A into the priority queue at distance zero.

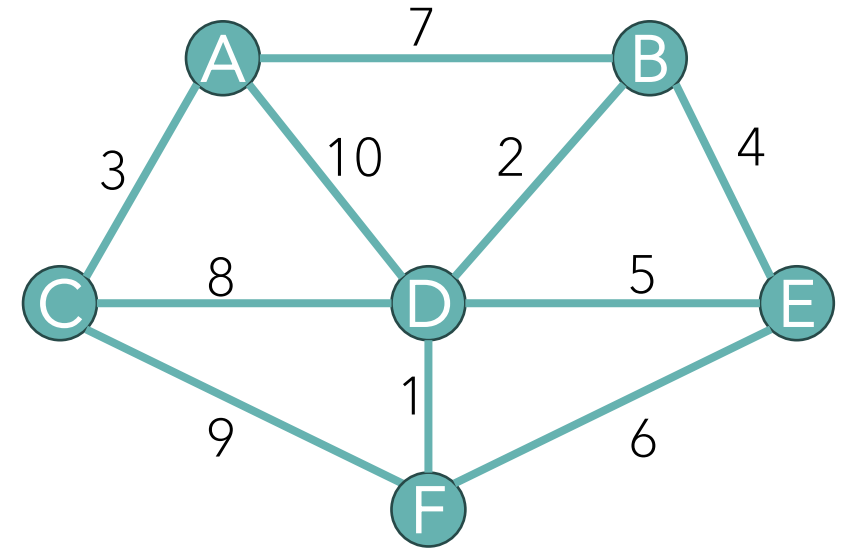


Node	Distance	Parent
A	0	n/a
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	

EXAMPLE RUN OF DIJKSTRA'S ALGORITHM

A comes out of the PQ, and we look at all of its neighbors.

Each one is added to the PQ if not already there, and has its priority (possibly) updated if not.

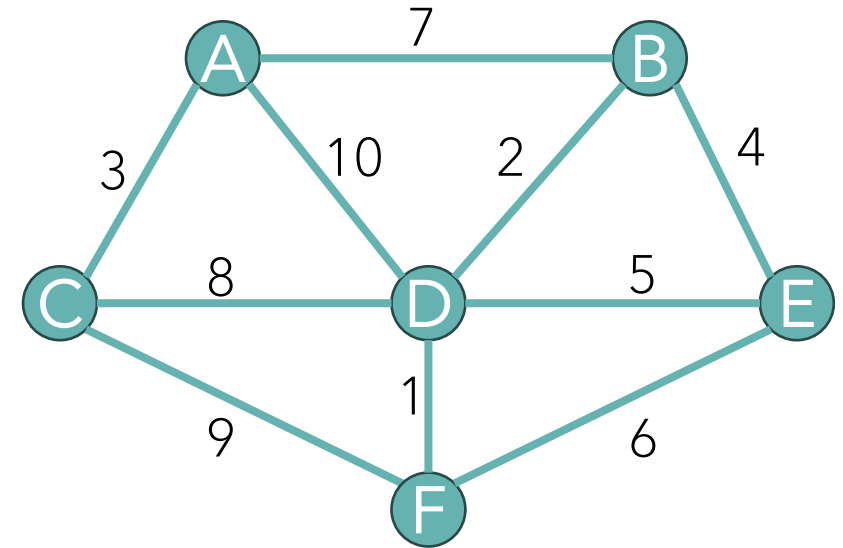


Node	Distance	Parent
A	0	n/a
B	7	A
C	3	A
D	10	A
E	∞	
F	∞	

EXAMPLE RUN OF DIJKSTRA'S ALGORITHM

We are now done with A – when we finish processing a node, its distance to the source is known to be the shortest possible.

Processing C, we need to add its own distance to the edge weights of its neighbors. Since $3 + 8 \geq 10$, we do not update the priority for D.

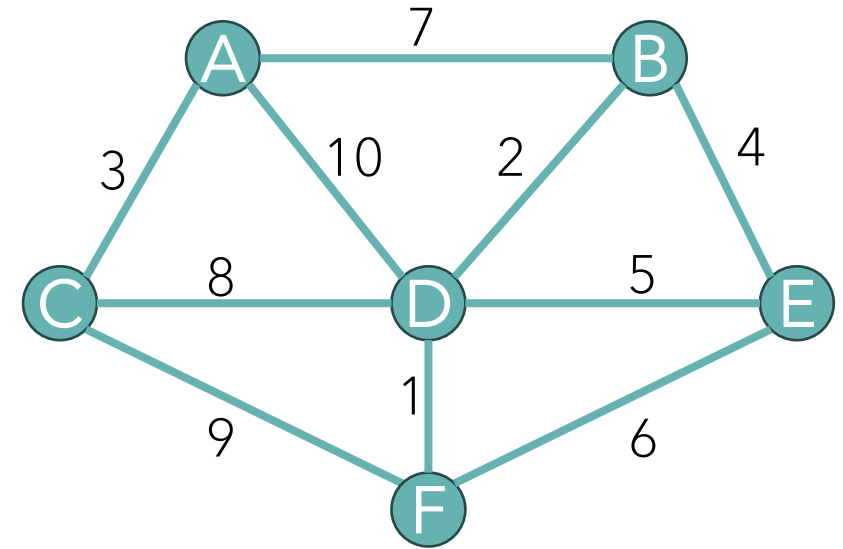


Node	Distance	Parent
A	0	n/a
B	7	A
C	3	A
D	10	A
E	∞	
F	12	C

EXAMPLE RUN OF DIJKSTRA'S ALGORITHM

C is finished; we move on to B.

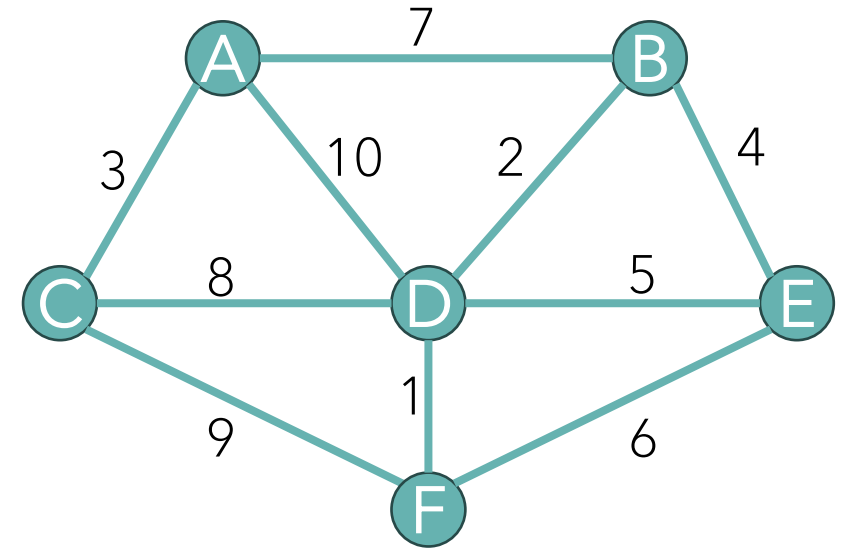
This time, the distance to D has decreased: $7 + 2 < 10$, so we update D's priority and parent.



Node	Distance	Parent
A	0	n/a
B	7	A
C	3	A
D	9	B
E	11	B
F	12	C

EXAMPLE RUN OF DIJKSTRA'S ALGORITHM

D is up next. It provides a shorter path to F, so we update F's priority. The path to E that goes through D is longer, so no update there.

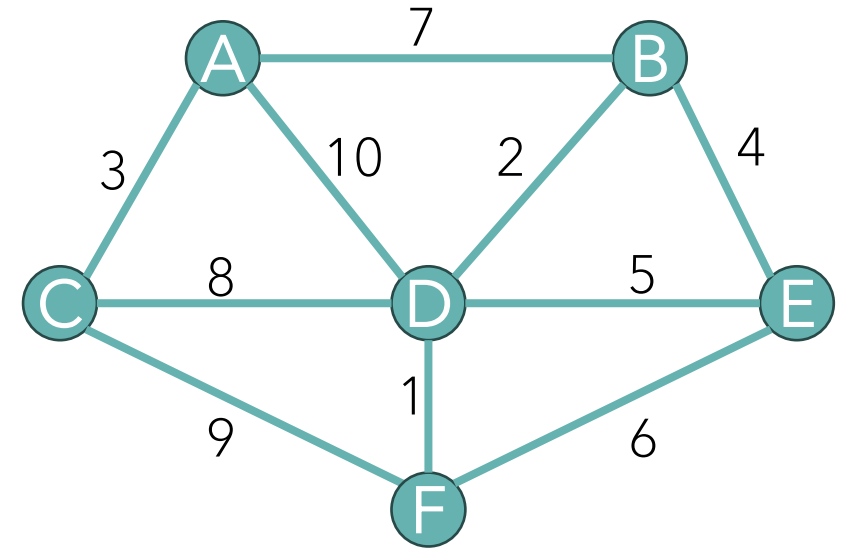


Node	Distance	Parent
A	0	n/a
B	7	A
C	3	A
D	9	B
E	11	B
F	10	D

EXAMPLE RUN OF DIJKSTRA'S ALGORITHM

F comes out of the priority queue here. At this point, we can stop - we know there will be no shorter path from A to F because we have explored every path out of A that has distance < 10 .

We read the path off by tracing parents back from F: The shortest path is F - D - B - A.



Node	Distance	Parent
A	0	n/a
B	7	A
C	3	A
D	9	B
E	11	B
F	10	D

DIJKSTRA'S ALGORITHM

Given: A graph (directed or not), a source v_s and a target v_t .

For every $v \in V$, $\text{parent}(v) = \text{NULL}$, $\text{dist}(v) = \infty$

Set $\text{dist}(v_s) = 0$ and $v_{cur} = v_s$

while $v_{cur} \neq v_t$:

for every uncompleted neighbor v_n of v_{cur} :

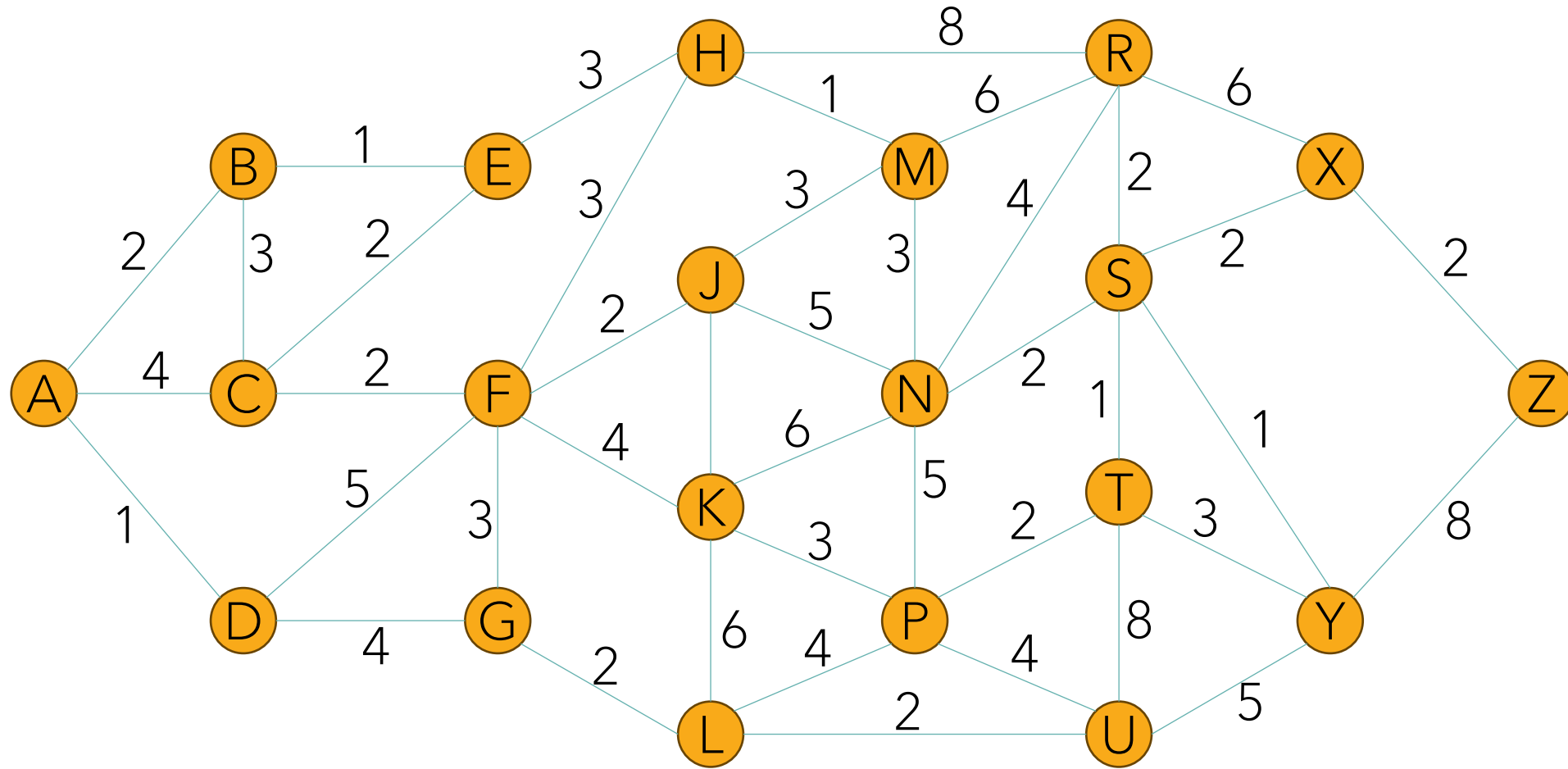
$d_n = \text{dist}(v_{cur}) + \text{weight}(v_{cur}, v_n)$

if $d_n < \text{dist}(v_n)$, set $\text{dist}(v_n) = d_n$

Mark v_{cur} as completed.

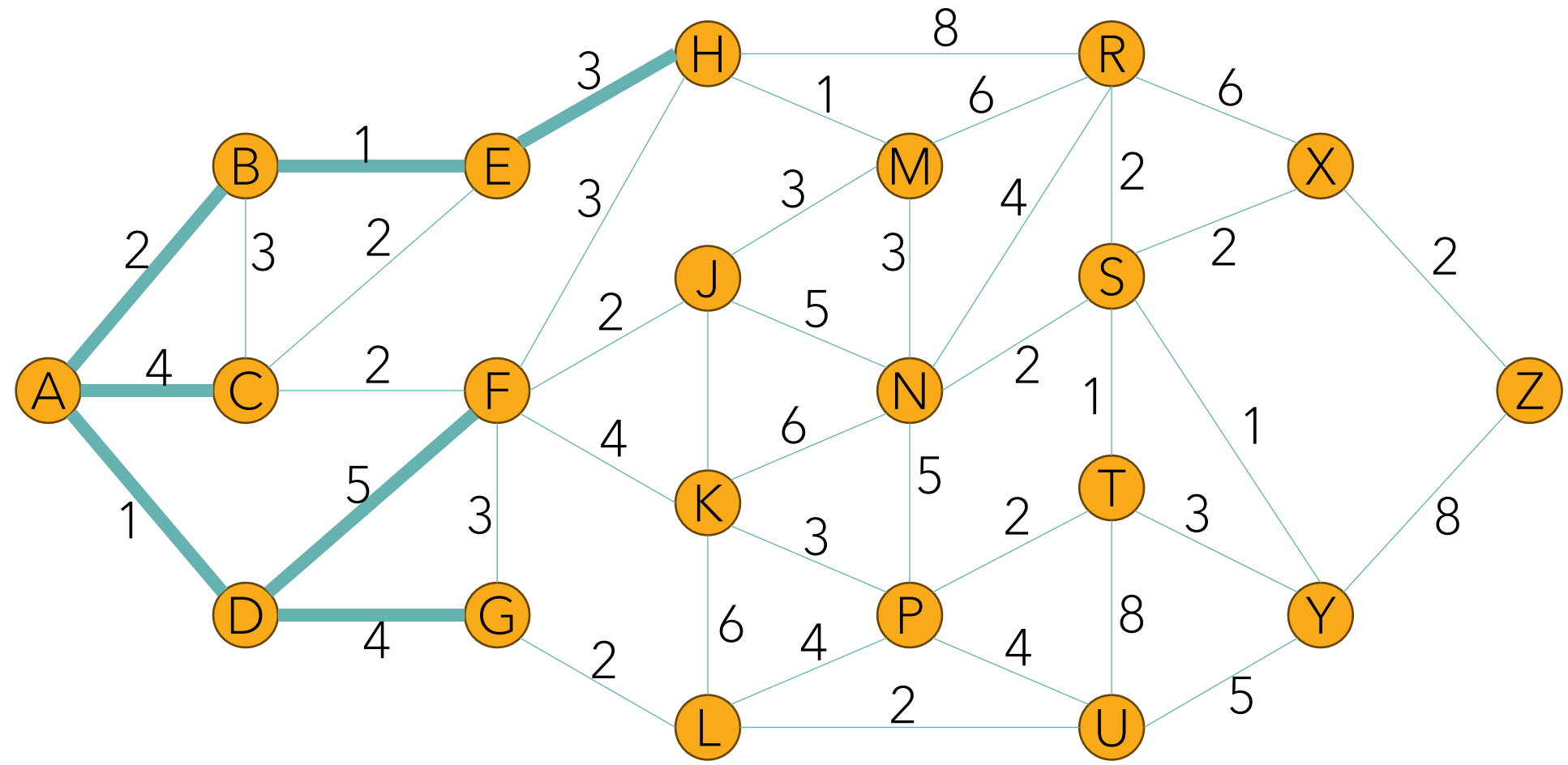
v_{cur} = the uncompleted vertex with lowest dist (maintained in PQ)

FROM A TO Z



DO NOW: Execute Dijkstra's algorithm, with source A and target Z.

FROM A TO Z



Node	Distance
A	0
B	2
C	4
D	1
E	3
F	6
G	5
H	6
J	8
K	10
L	7
M	7
N	
P	
R	14
S	
T	
U	
X	
Y	
Z	

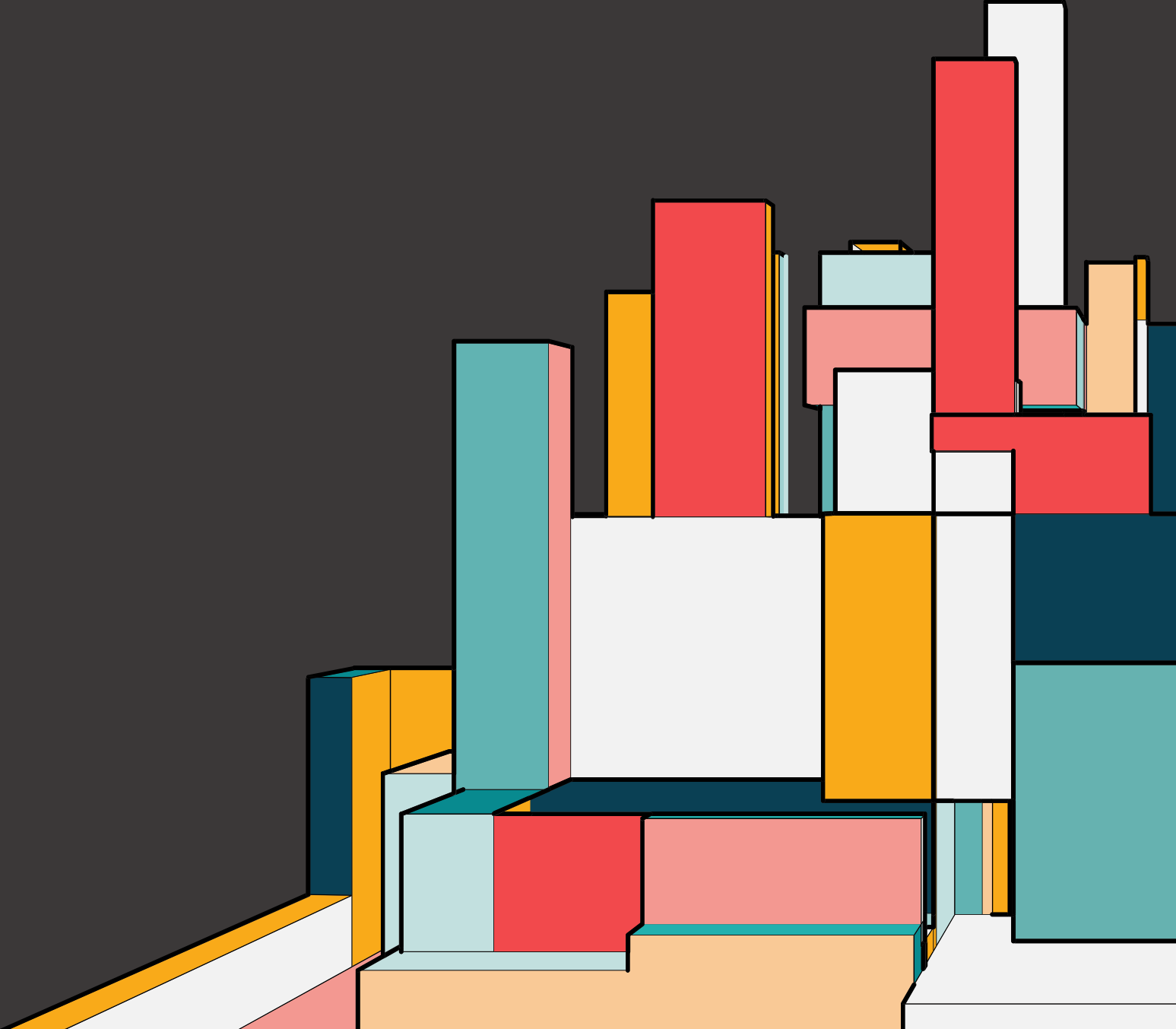
LIMITATIONS OF DIJKSTRA'S ALGORITHM

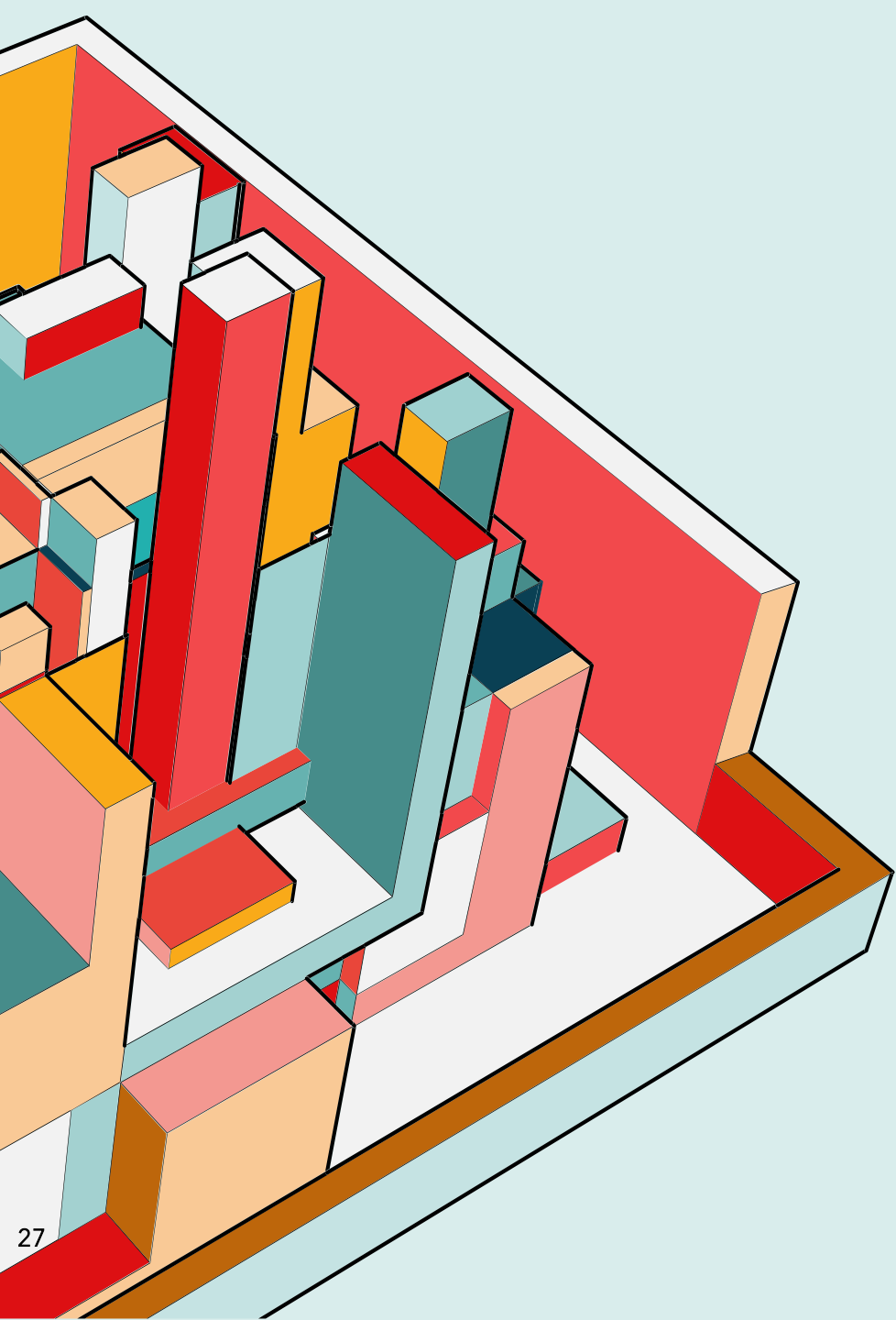
The best implementations of Dijkstra's algorithm run in $O(E + V \log V)$ time; no better single-source shortest paths algorithm is known.

But if you need to know about the shortest paths from multiple points, you can do better than just running Dijkstra from each vertex. (More on this later in the course.)

Also: Dijkstra's algorithm does not work if there are negative edge weights in the graph. (There are other algorithms that allow for this; again, we'll get there later in the course.)

QUICK BREAK





HUFFMAN CODES

BACK TO INFORMATION THEORY

The English alphabet has 26 letters. If we want to assign a unique bitstring to each one, what is the minimum number of bits that we need?

A: 5 bits, because $2^4 < 26$ but $2^5 > 26$.

But wait! You might argue that we CAN do it in four bits:

A = 0, B = 1, C = 00, D = 01, E = 10, F = 11, G = 000, ...

There are 31 non-empty bitstrings of up to four bits.

What's the problem?

001010111101101111101101001001001010101001001101010

What's the message there? How can you tell?

BACK TO INFORMATION THEORY

We need some way to tell where one symbol ends and the next one begins.

The obvious way to do that is *fixed-length* encoding – every symbol gets the same number of bits.

This is how all of the standard text encodings like ASCII (8^* bits) and Unicode (16^{**} bits) operate.

* Technically, ASCII is a seven-bit scheme. But we usually look at extended ASCII or UTF-8 these days, which are extensions of the original 7-bit system.

** Unicode really is strictly 16 bits, but it uses a code page system to swap out what symbol goes with certain bitstrings so that it can represent more than 2^{16} symbols.

DATA COMPRESSION

What if we need to reduce the number of bits a text document takes up?

If we want to devise a general system for compressing any text document, no fixed-length encoding system will help!

We could try a variable-length encoding scheme ... But how is that going to help?

A: Entropy!

ENTROPY == INFORMATION

Recall from our discussion of decision trees that the amount of information something has is precisely how surprising it is – more unlikely events tell us more than less likely ones.

We can apply this directly to data compression: Symbols that come up more often should be encoded using fewer bits.

In English, that means we want fewer bits for **e**, **t**, **a** and more bits for **q**, **j**, **z**.

Those of you that have done CAOS will recognize this as a variant on the general design principle “make the common case efficient”.

PROOF OF CONCEPT

Imagine for a moment we only have four letters in the alphabet: W, E, Q, X. We know that a fixed-length encoding scheme will need 2 bits per letter.

But let's say we also know that 60% of all symbols are W, 20% are Q, 10% are E, and 10% are X.

Let's try this encoding: W = 0, Q = 10, E = 110, X = 111

Notice that while W is shorter than the fixed-length encoding scheme, other symbols require more bits. This tradeoff is unavoidable.

What is the expected (i.e. average) number of bits per symbol that we will need to encode a message?

SUBMITTY POLL #2

What is the average number of bits per symbol in the variable-length scheme suggested?



PROOF OF CONCEPT

But let's say we also know that 60% of all symbols are W, 20% are Q, 10% are E, and 10% are X.

Let's try this encoding: W = 0, Q = 10, E = 110, X = 111

What is the expected number of bits per symbol that we will need to encode a message?

60% chance of 1 bit, 20% chance of 2 bits, 20% of 3 bits

$$(0.6)1 + (0.2)2 + (0.2)3 = 1.6 \text{ bits}$$

It works! On average, we'll use fewer bits in this system than a fixed-length scheme.

A ● ■
 B ■ ● ● ●
 C ■ ● ■ ●
 D ■ ● ●
 E ●
 F ● ● ■ ●
 G ■ ■ ●
 H ● ● ● ●
 I ● ●
 J ● ■ ■ ■
 K ■ ● ■
 L ● ■ ● ●
 M ■ ■
 N ■ ●
 O ■ ■ ■
 P ● ■ ■ ●
 Q ■ ■ ● ■
 R ● ■ ●
 S ● ● ●
 T ■

U ● ● ■
 V ● ● ● ■
 W ● ■ ■
 X ■ ● ● ■
 Y ■ ● ■ ■
 Z ■ ■ ● ●

1 ● ■ ■ ■ ■
 2 ● ● ■ ■ ■
 3 ● ● ● ■ ■
 4 ● ● ● ● ■
 5 ● ● ● ● ●
 6 ■ ● ● ● ●
 7 ■ ■ ● ● ●
 8 ■ ■ ■ ● ●
 9 ■ ■ ■ ■ ●
 0 ■ ■ ■ ■ ■

DISTINGUISHING SYMBOLS

We're still left with another problem: How might we design a variable-length encoding scheme where we can still look a stream of bits and be able to tell where each symbol ends and the next one begins?

Take a closer look at our recent scheme: $W = 0$, $Q = 10$, $E = 110$, $X = 111$

011110100011001101100010000111011010010

Can we determine which bits go with which symbols?

YES!

011110100011001101100010000111011010010

PREFIX CODES

$W = 0, Q = 10, E = 110, X = 111$

The reason this works is that none of the codewords are prefixes of another one. $W = 0$, and no other codewords start with 0.

Similarly, $Q = 10$ and nothing else starts with 10.

This allows us to read the bits left-to-right and, as soon as we see something that matches one of our symbols, immediately assign that symbol and start looking for the next one.

011110100011001101100010000111011010010

011110100011001101100010000111011010010

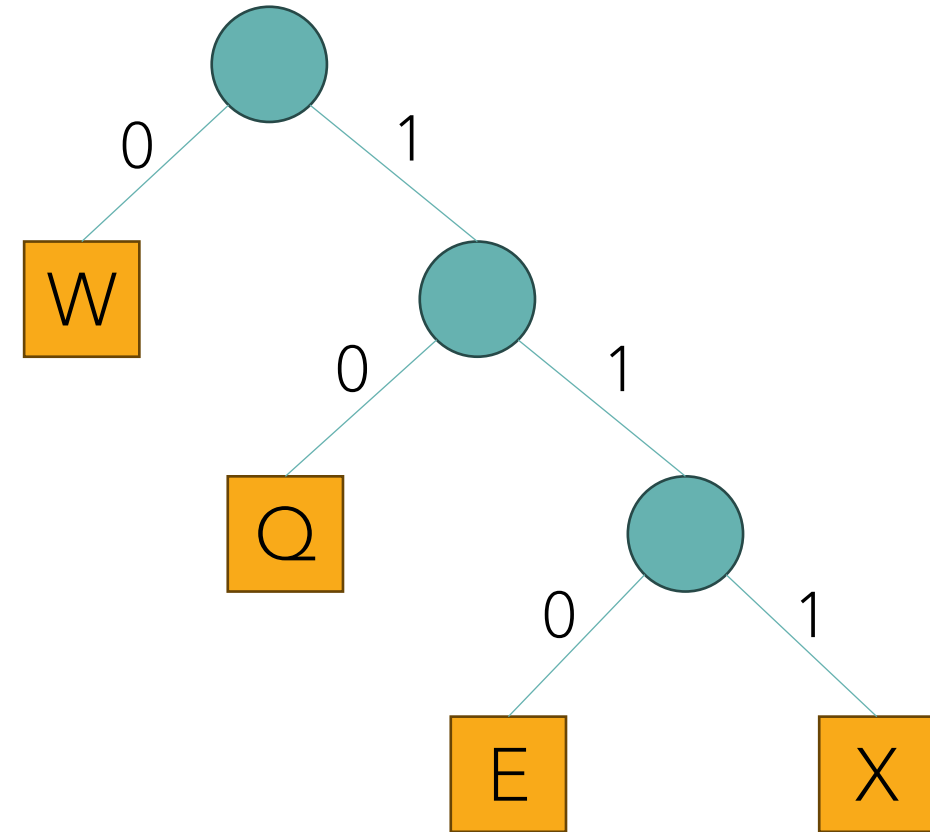
PREFIX CODES

A system which has this property is called a *prefix-free encoding*, or more commonly a *prefix code*.

Think carefully for a moment about the way this allows us to match symbols in an online (i.e. streaming) context.

What sort of data structure is best suited to storing our encoding scheme?

A tree! And specifically, a decision tree!



OPTIMIZING THE SYSTEM

Now that we know we can compress data in this fashion, an obvious next question is: What is the best way to do so in order to achieve maximum compression?

As you might expect in this chapter, the answer uses a greedy algorithm to construct the tree.

Take a moment and try to come up with an algorithm to generate the code tree.

CONSTRUCTING A CODE TREE

There are two main approaches to tree construction:

- Top-down: At each level, partition the symbols so that the probabilities for 0 and 1 are as even as possible. This is the approach of Robert Fano, an MIT professor. Claude Shannon himself came up with something similar.
- Bottom-up: Assign each symbol to a one-node tree, and then merge the lowest probability trees together at each step. This is the approach of David Huffman, one of Fano's graduate students.

They are both good, but in this case, the student outperformed the teacher. Only the latter one is optimal.

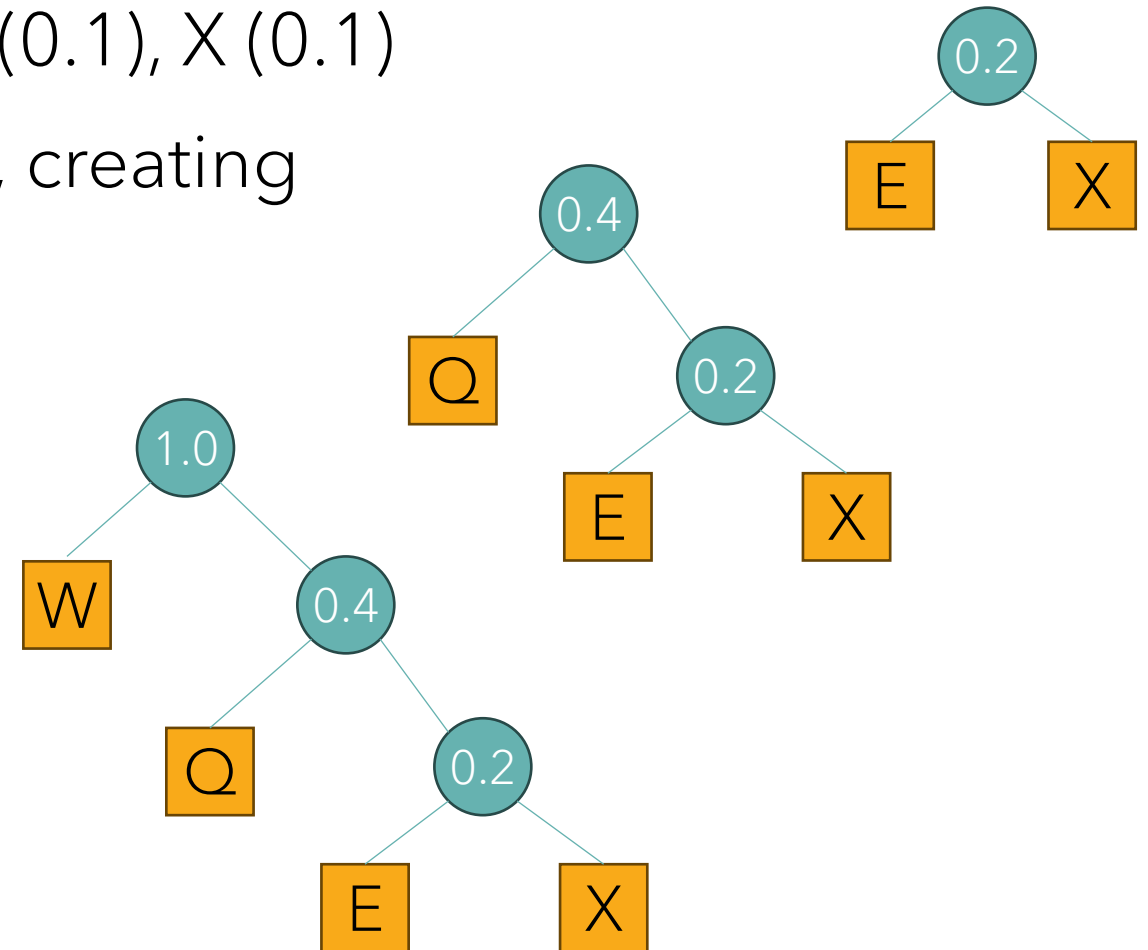
HUFFMAN CODES

Let's walk through the creation of a Huffman tree using our prior example: W (0.6), Q (0.2), E (0.1), X (0.1)

Our first step is to merge E and X, creating subtree EX with probability 0.2.

Next we merge Q with EX.

Finally, we merge W with QEX.



HUFFMAN'S ALGORITHM

- 1) Create n single-node trees, one per symbol. Assign each tree a weight equal to the frequency of its symbol.
- 2) Repeat until there is only a single tree with weight 1.0:
Take the two trees with the smallest weights. Merge these trees into a new one by making them the left and right subtrees of a single root node. Assign the new tree a weight equal to the sum of the weights of the trees that were merged.

Try it now on the following alphabet with five symbols:

J = 0.35, K = 0.1, Q = 0.2, X = 0.2, Z = 0.15

HUFFMAN TREE EXAMPLE

$J = 0.35, K = 0.1, Q = 0.2, X = 0.2, Z = 0.15$

First merge K & Z. $KZ = 0.25$

Next we merge Q & X. $QX = 0.4$.

Now we merge J & KZ. $JKZ = 0.6$

We're down to two trees, so merge them.

If $L = 0$ & $R = 1$...

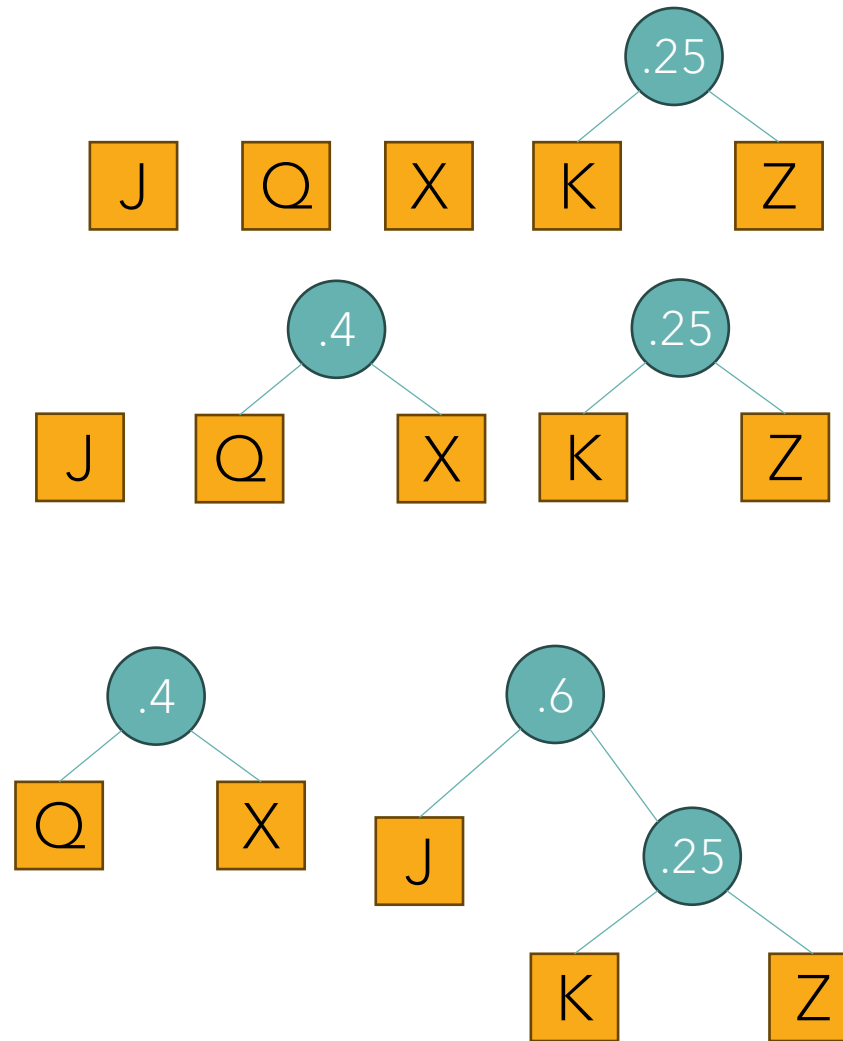
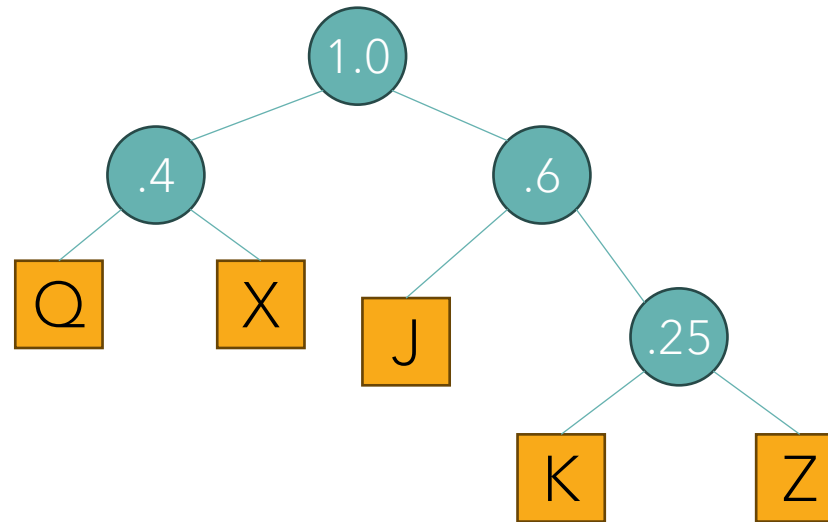
Q = 00

X = 01

J = 10

K = 110

Z = 111



BEFORE YOU LEAVE

